

# Práctica 2.1: Inicio de sesión seguro

Autor: Lex Cristofer Martinez Camarena

Proyecto: Sitio seguro

Repositorio: <https://github.com/Lexioor/ids-login-web>

Objetivo: implementar y demostrar validación del correo, protección de contraseñas y autenticación multifactor antes de conceder acceso.

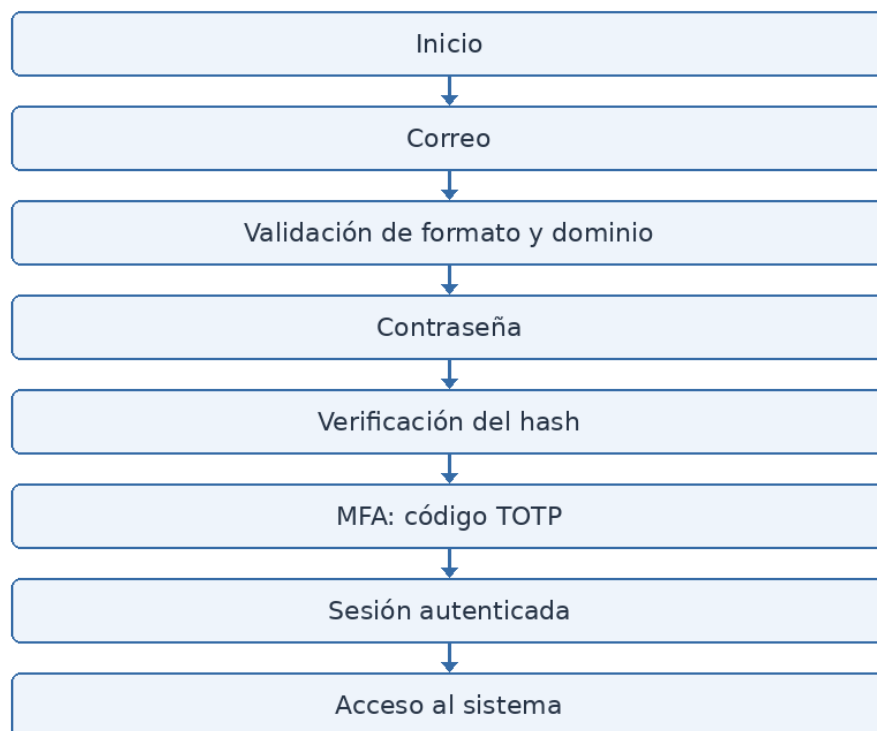
Tecnologías: Python, Flask, SQLite, Werkzeug (scrypt), TOTP y QR generado localmente.

Método: capturas de Chromium con el sitio funcionando y una base de datos aislada para pruebas. Las capturas de código se renderizaron directamente desde app.py. No se modificó la lógica de autenticación para obtener las evidencias.

Alcance: se verificó el dominio, no la propiedad del correo. El secreto QR se ocultó únicamente en la captura; los códigos mostrados corresponden a una cuenta temporal. El proyecto es académico y no constituye una garantía de seguridad absoluta.

# Flujo del módulo

Inicio → Correo → Validación → Contraseña → Verificación → MFA → Sesión → Acceso



Si el formato, el dominio, la contraseña o el segundo factor fallan, el flujo se detiene y no se concede acceso. Durante el registro, la cuenta se crea después de comprobar MFA.

# Código: validación del correo

validate\_email valida el formato y consulta MX, A o AAAA. Ante bloqueo de DNS directo utiliza DNS por HTTPS con TLS verificado. Existencia de dominio no equivale a propiedad del buzón.

app.py — def validate\_email — parte 1

```
47 def validate_email(value):
48     value = value.strip().lower()
49     if len(value) > 254 or not re.fullmatch(r'[a-z0-9.!#$%&\x27*+/?^_`{|}~-]+@[a-z0-9](?:[a-z0-9-]*[a-z0-9])?(?:\.[a-z0-9](?:[a-z0-9-]*[a-z0-9])?)+', value):
50         raise ValueError('Introduce un correo electrónico válido.')
51     if '..' in value.split('@')[0] or value.startswith('.') or value.split('@')[0].endswith('.'):
52         raise ValueError('Introduce un correo electrónico válido.')
53     domain = value.split('@')[1]
54     try:
55         # An SOA answer, including an enclosing zone, is insufficient: ask for
56         # records on the exact domain. DNS existence does not prove mailbox ownership.
57         for kind in ('MX', 'A', 'AAAA'):
58             try:
59                 if dns.resolver.resolve(domain, kind, lifetime=4):
60                     return value
61             except dns.resolver.NoAnswer:
62                 continue
63     except dns.resolver.NXDOMAIN:
64         raise ValueError('El dominio del correo no existe.') from None
65     except (dns.exception.DNSException, OSError):
66         # Cloud environments may disallow UDP DNS; use TLS-verified HTTPS
67         # through the platform proxy, without bypassing network restrictions.
68         try:
69             for kind in ('MX', 'A', 'AAAA'):
```

# Código: validación del correo

`validate_email` valida el formato y consulta MX, A o AAAA. Ante bloqueo de DNS directo utiliza DNS por HTTPS con TLS verificado. Existencia de dominio no equivale a propiedad del buzón.

app.py — def validate\_email — parte 2

```
70         url = 'https://cloudflare-dns.com/dns-query?' + urllib.parse.urlencode({'name':
domain, 'type': kind})
71         query = urllib.request.Request(url, headers={'Accept': 'application/dns-json'})
72         with urllib.request.urlopen(query, timeout=5) as response:
73             result = json.load(response)
74             if result.get('Status') == 3:
75                 raise ValueError('El dominio del correo no existe.')
76             if result.get('Status') != 0:
77                 raise OSError('DNS lookup failed')
78             if any(record.get('type') in (1, 15, 28) for record in result.get('Answer',
[])):
79                 return value
80         except (OSError, json.JSONDecodeError):
81             raise ValueError('No pudimos verificar el dominio. Inténtalo de nuevo.') from None
82         raise ValueError('El dominio no tiene registros de correo o dirección válidos.')
83
84
```

# Código: registro y hash de contraseña

register comprueba la longitud y la confirmación. generate\_password\_hash utiliza scrypt y salt; la contraseña en texto plano no se guarda.

app.py — def register — parte 1

```
151 def register():
152     error = None
153     if request.method == 'POST':
154         try:
155             email = validate_email(request.form.get('email', ''))
156             password = request.form.get('password', '')
157             if not 8 <= len(password) <= 128:
158                 raise ValueError('La contraseña debe tener entre 8 y 128 caracteres.')
159             if password != request.form.get('confirm', ''):
160                 raise ValueError('Las contraseñas no coinciden.')
161             with db() as conn:
162                 if conn.execute('SELECT id FROM users WHERE email=?', (email,)).fetchone():
163                     raise ValueError('No se pudo registrar este correo. Intenta iniciar
sesión.')
164                 # Store only the hash during the short enrollment; no account exists yet.
165                 session.clear()
166                 session.update(enrollment={'email': email, 'password':
generate_password_hash(password, method='scrypt'),
167                                     'secret':
base64.b32encode(secrets.token_bytes(20)).decode()}, expires=time.time() + 300)
168                 app.session_interface.regenerate(session)
169                 return redirect(url_for('mfa'))
170             except ValueError as e:
171                 error = str(e)
172         return render_template('auth.html', mode='register', error=error)
173
```

# Código: registro y hash de contraseña

register comprueba la longitud y la confirmación. generate\_password\_hash utiliza scrypt y salt; la contraseña en texto plano no se guarda.

app.py — def register — parte 2

174

# Código: verificación de credenciales

login busca la cuenta y usa check\_password\_hash. Si la contraseña es válida crea únicamente un desafío pendiente y redirige a MFA.

app.py — def login — parte 1

```
130 def login():
131     error = None
132     if request.method == 'POST':
133         try:
134             email = validate_email(request.form.get('email', ''))
135             if limited('login:' + email):
136                 raise ValueError('Demasiados intentos. Espera 5 minutos.')
137             with db() as conn:
138                 user = conn.execute('SELECT * FROM users WHERE email=?', (email,)).fetchone()
139                 if not user or not check_password_hash(user['password'],
request.form.get('password', '')):
140                     raise ValueError('Correo o contraseña incorrectos.')
141                 session.clear()
142                 session.update(pending=user['id'], expires=time.time() + 300)
143                 app.session_interface.regenerate(session)
144                 return redirect(url_for('mfa'))
145         except ValueError as e:
146             error = str(e)
147     return render_template('auth.html', mode='login', error=error)
148
149
```

# Código: cálculo y validación TOTP

totp implementa códigos de seis dígitos con intervalos de 30 segundos. verify\_totp admite una ventana limitada y rechaza códigos ya consumidos.

app.py — def totp — parte 1

```
85 def totp(secret, counter):
86     digest = hmac.new(base64.b32decode(secret), struct.pack('>Q', counter),
hashlib.shal).digest()
87     offset = digest[-1] & 15
88     return f'{{(struct.unpack(">I", digest[offset:offset + 4])[0] & 0x7fffffff) % 1000000:06d}}'
89
90
91 def verify_totp(secret, code, last=-1):
92     if not re.fullmatch(r'\d{6}', code):
93         return None
94     now = int(time.time()) // 30
95     for counter in (now - 1, now, now + 1):
96         if counter > last and hmac.compare_digest(totp(secret, counter), code):
97             return counter
98     return None
99
100
```

# Código: MFA y creación de sesión

La ruta mfa verifica el segundo factor. Solo entonces guarda uid en la sesión autenticada y permite el acceso al panel.

app.py — def mfa(): — parte 1

```
195 def mfa():
196     enrollment = session.get('enrollment')
197     pending = session.get('pending')
198     if (not enrollment and not pending) or session.get('expires', 0) < time.time():
199         session.clear()
200         return redirect(url_for('login'))
201     error = None
202     if request.method == 'POST':
203         identity = 'mfa:' + (enrollment['email'] if enrollment else str(pending))
204         if limited(identity):
205             error = 'Demasiados intentos. Espera 5 minutos y vuelve a iniciar sesión.'
206         else:
207             with db() as conn:
208                 user = None if enrollment else conn.execute('SELECT * FROM users WHERE id=?',
209 (pending,)).fetchone()
210                 counter = verify_totp(enrollment['secret'] if enrollment else user['secret'],
211 request.form.get('code', ''), -1 if enrollment else user['last_counter'])
212                 if counter is None:
213                     error = 'Código incorrecto o ya utilizado. Usa el código actual de tu app.'
214                 else:
215                     if enrollment:
216                         try:
217                             cursor = conn.execute('INSERT INTO
218 users(email,password,secret,last_counter) VALUES (?,?,,?)',
219 (enrollment['email'], enrollment['password'],
220 enrollment['secret'], counter))
221                             uid = cursor.lastrowid
```

## Código: MFA y creación de sesión

La ruta mfa verifica el segundo factor. Solo entonces guarda uid en la sesión autenticada y permite el acceso al panel.

```
app.py — def mfa(): — parte 2
218             except sqlite3.IntegrityError:
219                 session.clear()
220                 return redirect(url_for('login'))
221             else:
222                 cursor = conn.execute('UPDATE users SET last_counter=? WHERE id=? AND
last_counter<?', (counter, pending, counter))
223                 if cursor.rowcount != 1:
224                     abort(400)
225                 uid = pending
226                 session.clear()
227                 session.update(uid=uid, authenticated_at=time.time())
228                 app.session_interface.regenerate(session)
229                 return redirect(url_for('dashboard'))
230     return render_template('auth.html', mode='mfa', enrollment=enrollment, error=error)
231
232
```

# Validación del correo

Se envió un correo mal formado. El servidor rechazó la solicitud con «Introduce un correo electrónico válido». Se omitió la validación del navegador únicamente para comprobar también la del servidor.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

BIENVENIDO A SITIO SEGURO

### Inicia sesión.

Vuelve a tu espacio con tu correo y contraseña.

Introduce un correo electrónico válido.

Correo electrónico

correo-invalido

Contraseña

Introduce tu contraseña

Continuar

¿Aún no tienes cuenta? [Regístrate](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Rechazo de dominio inexistente

Se consultó el dominio reservado dominio.invalid mediante el flujo real de DNS. El servidor rechazó el correo porque el dominio no existe.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

BIENVENIDO A SITIO SEGURO

### Inicia sesión.

Vuelve a tu espacio con tu correo y contraseña.

El dominio del correo no existe.

Correo electrónico

evidencia@dominio.invalid

Contraseña

Introduce tu contraseña

Continuar

¿Aún no tienes cuenta? [Regístrate](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Política de contraseña

Una contraseña de tres caracteres fue rechazada. La política admite entre 8 y 128 caracteres y permite letras, números y símbolos.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

BIENVENIDO A SITIO SEGURO

### Crea tu cuenta.

Tu espacio empieza aquí. Usa tu correo y elige una contraseña segura.

La contraseña debe tener entre 8 y 128 caracteres.

Correo electrónico

lex.evidencia@gmail.com

Contraseña

Introduce tu contraseña

Entre 8 y 128 caracteres. Puedes usar letras, números, espacios y símbolos.

Confirmar contraseña

Repite tu contraseña

Configurar segundo factor

¿Ya tienes cuenta? [Inicia sesión](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Registro de contraseña

El formulario contiene correo, contraseña y confirmación. Los campos de contraseña están ocultos. Se utilizó una cuenta desechable de prueba; no se verificó la propiedad del buzón.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

BIENVENIDO A SITIO SEGURO

### Crea tu cuenta.

Tu espacio empieza aquí. Usa tu correo y elige una contraseña segura.

La contraseña debe tener entre 8 y 128 caracteres.

Correo electrónico

lex.evidencia@gmail.com

Contraseña

.....

Entre 8 y 128 caracteres. Puedes usar letras, números, espacios y símbolos.

Confirmar contraseña

.....

Configurar segundo factor

¿Ya tienes cuenta? [Inicia sesión](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Configuración del segundo factor

Tras registrar credenciales válidas, se solicita configurar TOTP en una app autenticadora. El QR y la clave se ocultaron en esta captura para no publicar el secreto. Se conserva el ingreso manual como alternativa.

</> Sitio seguro

INGENIERÍA · DESARROLLO WEB

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

PASO 02 / 02

### Un último paso.

Confirma tu identidad con tu app autenticadora.

QR oculto para proteger la clave de esta evidencia.

#### Configura tu segundo factor

Abre Google Authenticator, Microsoft Authenticator o Aegis. Agrega una cuenta y selecciona «Escanear código QR».

Después, introduce abajo el código de 6 dígitos que muestra tu app.

► ¿No puedes escanear? Usa la clave manual

No compartas este QR ni la clave. Solo se muestran durante el registro.

#### Código de verificación

0 0 0 0 0 0

Introduce los 6 dígitos de tu app. El código cambia cada 30 segundos.

Verificar y acceder

[Volver al inicio](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Registro completado

La cuenta se guardó solo después de confirmar el código del segundo factor. Esta pantalla demuestra el registro y acceso inicial completos.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

IDENTIDAD CONFIRMADA



**Ya estás dentro.**

Acceso exitoso. Has completado los dos factores de autenticación.

Sesión iniciada como  
**lex.evidencia@gmail.com**

Cerrar sesión



Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Validación de contraseña

Un intento con contraseña incorrecta se rechazó mediante un mensaje genérico: «Correo o contraseña incorrectos».

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

BIENVENIDO A SITIO SEGURO

### Inicia sesión.

Vuelve a tu espacio con tu correo y contraseña.

Correo o contraseña incorrectos.

Correo electrónico

lex.evidencia@gmail.com

Contraseña

Introduce tu contraseña

Continuar

¿Aún no tienes cuenta? [Regístrate](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Solicitud del segundo factor en el login

Después de validar correo y contraseña, se muestra el campo MFA. Todavía no se ha creado una sesión autenticada con acceso al panel.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

PASO 02 / 02

### Un último paso.

Confirma tu identidad con tu app autenticadora.

Código de verificación

0 0 0 0 0 0

Introduce los 6 dígitos de tu app. El código cambia cada 30 segundos.

Verificar y acceder →

[Volver al inicio](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Código MFA incorrecto

Se introdujo un código numérico de seis dígitos que no coincide con ningún código aceptado. El sistema rechazó el segundo factor y mantuvo al usuario fuera del panel.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

PASO 02 / 02

### Un último paso.

Confirma tu identidad con tu app autenticadora.

Código incorrecto o ya utilizado. Usa el código actual de tu app.

Código de verificación

000000

Introduce los 6 dígitos de tu app. El código cambia cada 30 segundos.

Verificar y acceder



[Volver al inicio](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Código MFA actual

Se introdujo un código TOTP válido, generado con el secreto de la cuenta de prueba y la hora actual. La evidencia siguiente documenta su aceptación. Esta comprobación usa TOTP automatizado, no un teléfono físico.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

PASO 02 / 02

### Un último paso.

Confirma tu identidad con tu app autenticadora.

Código de verificación

408467

Introduce los 6 dígitos de tu app. El código cambia cada 30 segundos.

Verificar y acceder→

[Volver al inicio](#)

Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Código MFA correcto y acceso exitoso

Después de enviar el código correcto, el navegador llegó a /panel y mostró «Acceso exitoso». Correo y contraseña por sí solos no concedieron este acceso.

PROYECTO WEB / AUTENTICACIÓN

## Sitio seguro

Registro e inicio de sesión con verificación en dos pasos.

IDENTIDAD CONFIRMADA



**Ya estás dentro.**

Acceso exitoso. Has completado los dos factores de autenticación.

Sesión iniciada como  
**lex.evidencia@gmail.com**

Cerrar sesión



Cuida tu privacidad: no compartas tu contraseña ni tus códigos.

CORREO → CONTRASEÑA → MFA → ACCESO

# Resultados y conclusiones

Se obtuvieron evidencias de todos los puntos solicitados: validación del correo, dominio inexistente, registro y verificación de contraseña, solicitud MFA, código incorrecto, código correcto y acceso exitoso.

Se ejecutaron seis pruebas automatizadas con resultado satisfactorio. Cubren el flujo de registro/login, hash scrypt, rechazos de entradas inválidas, CSRF, límite de intentos, vector estándar TOTP, DNS por HTTPS y QR con caducidad.

La autenticación requiere ambos factores. Las sesiones son del lado del servidor; se renueva su identificador al autenticar, se limita su duración y se rechaza reutilizar un código consumido.

Para producción se requiere HTTPS, servidor WSGI, verificación de propiedad del correo, recuperación segura, cifrado gestionado de secretos TOTP y límites adicionales por IP. Estas mejoras no se presentan como implementadas.